

RAG

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

지식이 많이 필요한 자연어 처리 문제를 위한 검색 증강 생성

1. 개요

문제의식

기존의 대규모 사전학습 언어 모델은 학습 과정에서 많은 사실 지식을 파라미터 안에 저장함.

예를 들어 모델이 **프랑스의 수도는 파리** 라는 사실을 학습했다면, 이 정보는 별도의 데이터베이스에 저장되는 것이 아니라 **신경망 파라미터 전체에 분산된 형태로 저장됨**.

= **Parametric Memory (파라미터 기반 기억)** 이라고 부름.

하지만 이런 방식에는 몇 가지 문제가 있음.

- 새로운 지식을 추가하거나 기존 지식을 수정하기 어려움
- 모델이 어떤 정보를 근거로 답변했는지 확인하기 어려움
- 잘못된 내용을 사실처럼 생성하는 **환각** 문제가 존재함
- 지식이 많이 필요한 문제에서는 모델 내부 지식만으로 충분하지 않을 수 있음

이를 해결하기 위해 논문에서는 언어 모델 외부에 **검색 가능한 지식 저장소**를 추가함.

= **Non-parametric Memory (비파라미터 기반 기억)** 이라고 부름.

RAG

따라서 RAG의 핵심 구조는 다음과 같음.

RAG = Parametric Memory + Non-parametric Memory

역할	사용 모델
질문과 관련된 문서 검색	DPR
답변 생성	BART-large
외부 지식 저장소	Wikipedia
벡터 검색	FAISS

기존 검색 기반 모델과의 차이

이전에도 REALM, ORQA처럼 검색을 사용하는 모델은 존재했지만 주로 **추출형 질의응답 (Extractive QA)**에 사용됨.

** 추출형 QA: 검색한 문서 안에서 정답 부분을 그대로 찾아서 가져오는 방식

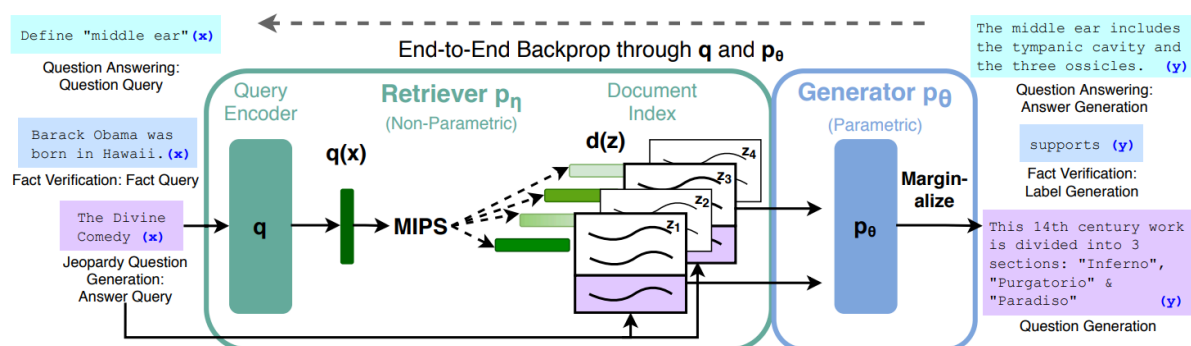
ex) 문서: Barack Obama was born in Hawaii.

질문: Barack Obama was born where?

→ 문서에서 **Hawaii** 를 직접 추출

반면 RAG는 **생성형 모델**: 검색한 문서를 참고하지만 최종 답변은 Generator가 직접 생성
따라서 검색 문서 안에 정답 문자열이 그대로 존재하지 않아도 문서가 제공하는 단서를 활용
해서 정답을 생성할 수 있음.

2. RAG 구조



$$x \rightarrow q(x) \rightarrow \text{Retriever} \rightarrow z_1, z_2, \dots, z_K \rightarrow \text{Generator} \rightarrow y$$

기호	의미
x	입력 질문 or 입력 문장
z	검색된 문서

기호	의미
y	최종 출력 문장
y_i	출력 문장의 i 번째 토큰
$y_{1:i-1}$	현재 토큰 이전에 생성된 토큰들
$p_\eta(z x)$	입력 x 에 대해 문서 z 가 관련 있을 확률
$p_\theta(y_i x, z, y_{1:i-1})$	질문, 문서, 이전 토큰이 주어졌을 때 다음 토큰을 생성할 확률

질문 입력 → 질문을 벡터로 변환 → Wikipedia에서 관련 문서 검색 → 검색 문서와 질문을 BART에 입력 → 답변 생성

이때 검색 결과 1등 문서 하나만 사용하는 것이 아니고 상위 K개의 문서를 가져온 뒤

문서 A를 사용했을 때의 답변 확률
문서 B를 사용했을 때의 답변 확률
문서 C를 사용했을 때의 답변 확률

모두 고려하여 최종 답변 확률을 계산

⇒ **Marginalization, 주변화**

3. RAG 모델

논문에서는 검색 문서를 어느 단위로 활용하는가에 따라 RAG를 두 가지 방식으로 제안함.

- RAG-Sequence
- RAG-Token

(1) RAG-Sequence

하나의 문서가 답변 전체 생성에 영향을 준다고 가정

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y|x, z)$$

토큰 단위로 풀어 쓰면: $p_{\text{RAG-Sequence}}(y|x) \approx$

$$\sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) \prod_{i=1}^N p_\theta(y_i|x, z, y_{1:i-1})$$

- $p_\eta(z|x)$: 질문 x 에 대해 문서 z 가 관련 있을 확률
- $p_\theta(y|x, z)$: 해당 문서를 참고했을 때 최종 답변 y 가 생성될 확률

⇒ $p_\eta(z|x)p_\theta(y|x, z)$: 이 문서를 검색하고 이 문서를 바탕으로 해당 답변이 나올 가능성

이 값을 상위 K개 문서에 대해 모두 더함.

= 문서 하나를 기준으로 문장 전체 확률을 계산한 뒤, 여러 문서의 결과를 마지막에 합침



왜 ≈인가

원래 정확하게 계산하려면 Wikipedia의 모든 문서에 대해 확률을 계산해야 함.
하지만 논문에서는 Wikipedia를 약 2,100만 개 passage로 분할함.

모든 문서를 계산하는 것은 현실적으로 불가능하기 때문에 Retriever가 찾은
Top-K 문서만 사용함.

따라서 정확한 값이 아니라 근사값임!



문장 확률이 곱으로 표현되는 이유

언어 모델은 문장을 한 번에 생성하지 않고 토큰을 하나씩 생성함.

따라서 $p_{\theta}(y|x, z) = \prod_{i=1}^N p_{\theta}(y_i|x, z, y_{1:i-1})$ 가 됨

ex) 정답이 Barack Obama 라면 $P(\text{Barack Obama}) = P(\text{Barack}) \times P(\text{Obama} | \text{Barack})$ 와 같은 방식

(2) RAG-Token

각 토큰을 생성할 때 서로 다른 문서를 활용

$$p_{\text{RAG-Token}}(y|x) \approx \prod_{i=1}^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

- RAG-Sequence는 $\sum_z \prod_i$
- RAG-Token은 $\prod_i \sum_z \Rightarrow$ **합과 곱의 순서가 반대**

RAG-Token에서는 토큰 하나를 생성할 때마다 $\sum_z p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$ 를 계산

= 지금 이 단어를 생성할 때 어떤 문서가 가장 도움이 되는가?를 토큰마다 다시 고려하는 구조

ex) Hemingway wrote The Sun Also Rises and A Farewell to Arms 라는 문장을 생성한다고 하면,

- The Sun Also Rises 생성 → 문서 A의 영향이 큼

- **A Farewell to Arms** 생성 → 문서 B의 영향이 큼 처럼 여러 문서를 조합할 수 있음.

*RAG-Sequence vs RAG-Token

구분	RAG-Sequence	RAG-Token
문서 활용 단위	문장 전체	토큰
문서 선택	답변 전체에서 동일한 문서를 가정	토큰마다 다른 문서 활용 가능
수식	$\sum z \prod i$	$\prod i \sum z$
장점	전체적으로 일관된 생성	여러 문서 정보 결합에 유리
직관	이 답변 전체에 적합한 문서는?	지금 이 단어에 적합한 문서는?

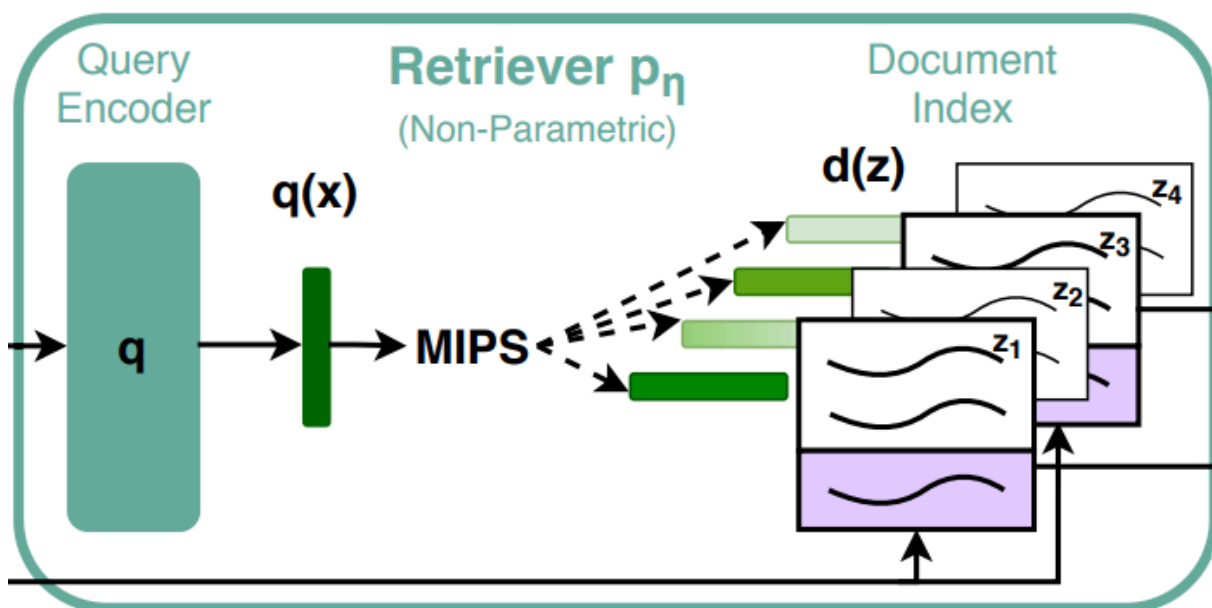
분류 문제에서는 두 모델이 동일함

분류 문제에서는 출력이 하나의 class token임.

N=10이 되면 출력 토큰이 하나이므로 RAG-Sequence와 RAG-Token의 차이가 사라짐.

따라서 FEVER 같은 분류 문제에서는 두 모델이 동일함.

4. Retriever: DPR



RAG의 검색기: DPR - Dense Passage Retriever

검색 확률

$$p_{\eta}(z|x) \propto \exp(d(z)^{\top} q(x))$$

- 내적값이 높은 문서일수록 높은 확률을 갖도록 함
- 실제로는 다음과 같은 Softmax 구조 : $p_{\eta}(z|x) = \frac{\exp(d(z)^{\top} q(x))}{\sum_{z'} \exp(d(z')^{\top} q(x))}$
- 논문에서는 분모를 생략하고 비례 기호 $\propto$ 로 표현

- **Query Encoder:** 질문 x 를 BERT 기반 Query Encoder에 넣어 벡터로 변환
 - $q(x) = BERT_q(x)$
- **Document Encoder:** Wikipedia 문서 z 를 BERT 기반 Document Encoder로 벡터화
 - $d(z) = BERT_d(z)$

= 질문과 문서를 같은 벡터 공간에 표현
- **문서와 질문의 유사도:** 질문과 문서의 관련성은 두 벡터의 **내적**으로 계산
 - $d(z)^{\top} q(x)$
 - 내적값이 클수록 질문과 문서가 관련 있다고 판단

Bi-Encoder 구조

DPR: 질문과 문서를 각각 독립적인 Encoder로 처리하는 **Bi-Encoder** 구조

$$x \rightarrow BERT_q \rightarrow q(x)$$

$$z \rightarrow BERT_d \rightarrow d(z)$$

- Wikipedia 문서 벡터를 미리 계산해둘 수 있음
- 새로운 질문이 들어오면 질문 벡터만 계산한 뒤 기존 문서 벡터와 비교하면 됨.

MIPS

Maximum Inner Product Search: Retriever는 내적값이 가장 큰 문서를 찾아야 한다

Wikipedia 문서가 매우 많기 때문에 모든 문서를 직접 비교하지 않고 FAISS를 이용한 근사 검색을 사용

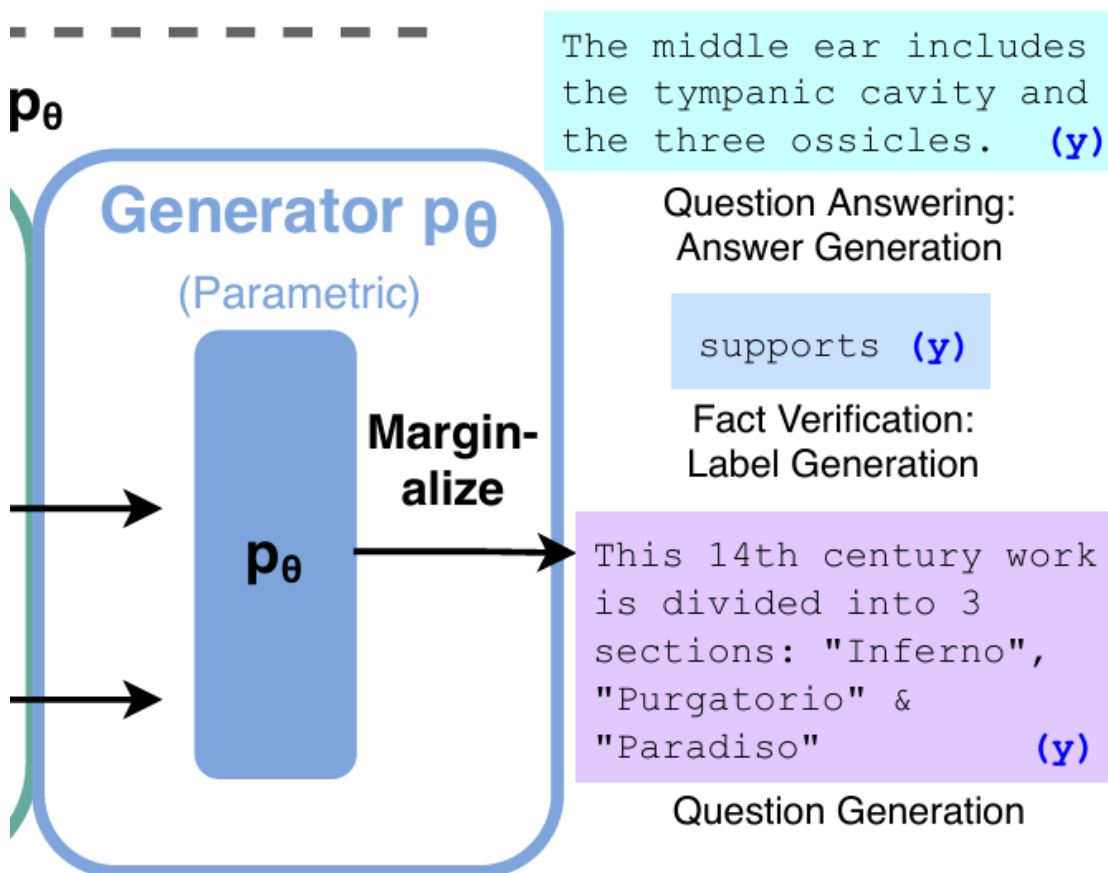
Retriever 초기화

Retriever를 처음부터 학습한 것은 아니고 Natural Questions와 TriviaQA에서 학습된 기존 DPR Retriever로 초기화함.

따라서 처음부터 어느 정도 관련 문서를 찾는 능력을 가지고 시작함.

5. Generator: BART

생성기는 BART-large 사용



Generator는 다음 확률을 계산 : $p_\theta(y_i|x, z, y_{1:i-1})$

= 질문 x , 검색된 문서 z , 지금까지 생성한 토큰들이 주어졌을 때 다음 토큰 y_i 를 생성할 확률

질문과 문서를 입력하는 방법

복잡한 결합 구조를 사용하지 않고 질문과 검색 문서를 단순히 이어 붙임.

ex) Who wrote The Divine Comedy? The Divine Comedy was written by Dante Alighieri ...

이 전체 내용을 BART Encoder에 입력하고 최종 답변을 생성

Parametric Memory

BART의 파라미터 θ 안에 저장된 지식을 의미

Non-parametric Memory

Wikipedia 문서와 이를 벡터화한 Document Index

즉 RAG는 **모델 내부 기억 + 외부 검색 가능한 기억**을 동시에 사용하는 구조

6. Training

RAG는 Retriever에게 “이 문서를 검색해야 함” 같은 정답 문서 라벨을 직접 주지 않음
학습 데이터에는 기본적으로 (x_j, y_j) 만 존재함.

ex)

- 입력 : Who wrote The Divine Comedy? $\rightarrow x_j$
- 정답 : Dante Alighieri $\rightarrow y_j$

어떤 Wikipedia 문서를 검색해야 하는지는 알려주지 않음.

학습 목적 함수

$\sum_j -\log p(y_j|x_j)$ 를 최소화 = **Negative Marginal Log-Likelihood**

의미

모델이 정답에 높은 확률을 주도록 학습하는 것

- 정답 확률이 $p(y|x)=0.9$ 라면 loss가 작음.
- 반대로 $p(y|x)=0.01$ 이라면 loss가 크게 나옴.

따라서 학습을 통해 정답 생성 확률을 높임.

왜 Marginal이라는 표현을 쓰는가

학습 데이터에서는 어떤 문서 z 가 정답 문서인지 알 수 없음.

즉 z 는 직접 관찰할 수 없는 **잠재 변수**임.

따라서 문서를 직접 하나 선택하는 대신 $p(y|x) = \sum_z p(z|x)p(y|x, z)$ 처럼 모든 후보 문서의 가능성을 합침.

이렇게 잠재 변수를 합으로 없애는 과정을 **Marginalization**이라고 함.

Retriever도 같이 학습되는 이유

어떤 문서를 검색했을 때 Generator가 정답을 잘 생성한다면 $p(z|x)p(y|x, z)$ 값이 커짐.

학습 과정에서 이 신호가 Retriever의 Query Encoder까지 전달됨.

따라서 Retriever는 별도의 정답 문서 supervision 없이도 어떤 문서를 검색해야 최종 정답을 만들기 쉬운가를 학습하게 됨.

구성요소	학습 여부
BART Generator	학습함
DPR Query Encoder	학습함
DPR Document Encoder	고정함
Wikipedia Index	고정함

** Document Encoder까지 학습하면 Wikipedia의 모든 문서 벡터가 바뀌기 때문에 전체 index를 계속 새로 구축해야 함. → 비용이 매우 크기 때문에 논문에서는 Document Encoder는 고정함.

7. Decoding

테스트 시에는 가장 확률이 높은 최종 답변 y 를 찾아야 함: $\arg \max_y p(y|x)$

RAG-Token과 RAG-Sequence의 확률 구조가 다르기 때문에 디코딩 방식도 다름.

(1) RAG-Token

토큰 단위 확률: $p'_\theta(y_i|x, y_{1:i-1}) = \sum_z p_\eta(z|x)p_\theta(y_i|x, z, y_{1:i-1})$

여러 문서에서 나온 다음 토큰 확률을 하나로 합칠 수 있기 때문에 일반적인 Seq2Seq 모델처럼 **Beam Search**를 그대로 적용할 수 있음.

(2) RAG-Sequence

$\sum_z p(z|x) \prod_i p(y_i|x, z, y_{<i})$ 구조이기 때문에 일반적인 Beam Search 하나만으로 계산하기 어려움.

그래서 두 가지 디코딩 방식을 제안함.

- **Thorough Decoding**

- 각 문서마다 Beam Search를 따로 수행

z1 → Beam Search

z2 → Beam Search

z3 → Beam Search

- 각 문서에서 나온 모든 후보 문장을 모아 후보 집합 Y를 만들고 → 각 후보 문장에 대해 모든 문서에서 다시 확률을 계산한 뒤 합함.
- 정확하지만 계산량이 큼.

- **Fast Decoding**

- 특정 문서의 Beam Search에서 후보 문장 y가 등장하지 않았다면

$p_{\theta}(y|x, z_i) \approx 0$ 이라고 가정

= 해당 문서에서는 그 문장의 가능성이 거의 없다고 보고 추가 계산을 생략함.

- Thorough Decoding보다 빠름.

8. 실험

논문에서는 RAG가 특정 질의응답 문제에만 효과적인 구조가 아니라는 점을 확인하기 위해 **질의응답, 문장 생성, 사실 검증** 등 여러 지식 집약적 자연어 처리 과제에서 실험함.

외부 지식으로는 **2018년 12월 Wikipedia**를 사용했으며, 문서를 100단어 단위로 나눠 약 **2,100만 개의 passage**를 구축

Open-domain Question Answering

RAG는 외부 검색 없이 모델 내부 지식만 사용하는 T5와 기존 검색 기반 모델인 REALM, DPR보다 전반적으로 높은 성능을 기록함.

특히 Natural Questions에서 RAG-Sequence가 44.5 EM을 기록하며 당시 최고 수준의 성능을 보임.

또한 검색 문서에 정답 문자열이 직접 포함되지 않은 경우에도 11.8%의 정답률을 기록함.

→ 단순히 문서에서 정답을 추출하는 것이 아니라, 검색된 문서의 단서와 모델 내부 지식을 결합해 답을 생성할 수 있음을 보여줌.

Abstractive QA

자연스러운 문장 형태의 답변을 생성하는 능력을 평가함.

RAG-Sequence가 BART보다 BLEU-1과 ROUGE-L 모두 약 2.6점 향상됨.

→ 검색된 외부 지식을 활용하면 순수 생성 모델보다 더 정확하고 사실적인 답변 생성이 가능함.

Jeopardy Question Generation

특정 인물이나 개념을 입력으로 주고, 해당 대상에 대한 사실 기반 질문을 생성하는 과제, RAG-Token이 가장 높은 성능을 기록함.

연구진은 RAG-Token이 토큰마다 서로 다른 문서의 정보를 활용할 수 있기 때문이라고 분석

Fact Verification

FEVER 데이터셋을 이용해 주어진 주장이

- Supports
- Refutes
- Not Enough Information 중 어디에 해당하는지 분류함.

RAG는 정답 evidence 문서를 직접 알려주지 않았음에도 높은 성능을 기록함.

→ 최종 정답 supervision만으로도 Retriever가 관련 근거 문서를 학습할 수 있음을 보여줌.

9. 추가 분석

Retriever 학습 효과

- Retriever를 고정했을 때보다 task에 맞게 함께 학습했을 때 대부분의 과제에서 성능이 향상됨.
 - **Generator뿐 아니라 Retriever까지 함께 fine-tuning하는 것이 중요함.**
- 전통적인 키워드 기반 검색 방식인 BM25와 비교했을 때도 대부분의 질의응답 과제에서 학습된 DPR이 더 높은 성능을 보임.
- FEVER에서는 entity와 단어 자체의 일치가 중요해 BM25가 더 좋은 결과를 보임.

Index Hot-Swapping

- 2016년과 2018년 Wikipedia index를 각각 사용해 국가 지도자 관련 질문을 평가함.
- 최신 index로 교체하면 Generator를 다시 학습하지 않아도 해당 시점의 최신 정보를 반영할 수 있었음.

→ RAG는 모델 자체를 재학습하지 않고 외부 지식 저장소만 교체해 지식을 업데이트할 수 있음.

검색 문서 수의 영향

검색하는 문서 수 KK를 늘린다고 항상 성능이 좋아지는 것은 아니었음.

- RAG-Sequence는 문서 수가 증가할수록 QA 성능이 전반적으로 향상됨
- RAG-Token은 약 K=10 근처에서 가장 좋은 성능을 보임

→ 검색 문서를 너무 많이 사용하는 경우 불필요한 정보가 오히려 노이즈가 될 수 있음을 보여줌.